16 ~~Sep~~ 2025 15 Oct 2026

Priority Queues

→ Sometimes there are processes / items in queues that need to be considered based on some priority order.

→ We don't need a full BST / RBT type data structure in this case.

→ We just need insert / Build / Max and Delete Max

We can consider a sorting algorithm if we have such a data structure. We can sort any set of keys by building the priority queue and successively deleting max.

For example:

Array: Insert is $O(1)$
Delete Max is $O(n)$

Sorted array: Insert is $O(n)$
Delete Max is $O(1)$

AVL tree: Both insert and delete max is $O(\log n)$

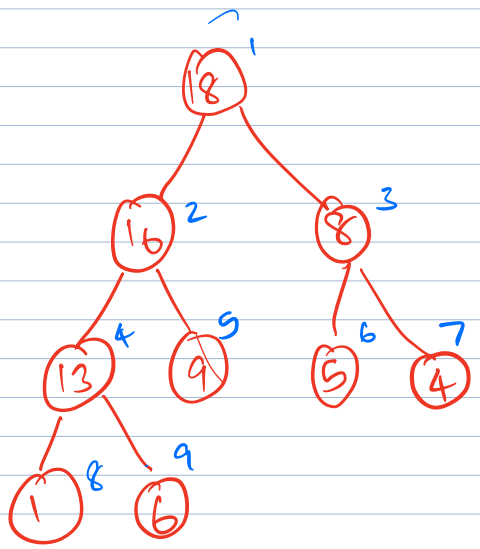
Want a simpler data structure for priority queue.

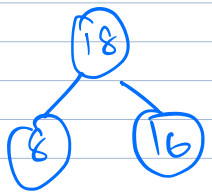
Binary Heaps

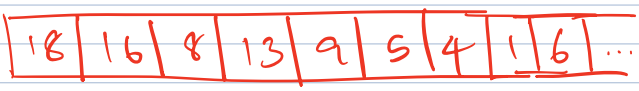
We will talk about max heaps, but everything translates to min heaps also.

Structural Property

Heap is a complete binary tree, except possibly the last level.







The structure helps in the maintenance of the data structure.

We store the entire thing as an array.

Note: There is a bijection between arrays and binary trees of this structure.

The tree structure is implicit.

Given n nodes, what is the height of such a heap?

Height $= \lfloor \log n \rfloor$

Given height h, what is the upper / lower bound on the number of keys in the heap?

$$2^h \leq n \leq 2^{h+1} - 1$$

Given node A[i], how can we access the children of this node?

Children of A[i]

Left child : A[2i]

Right child : A[2i+1]

Given node A[i], how can we access the parent?

Parent : $A\left[\left\lfloor \frac{i}{2} \right\rfloor\right]$

Some consequences of the max heap property.

→ largest element will be at the root.

→ Second largest element will be a child of the root. Else it violates heap property with its parent.

→ Third largest element will be either child or grandchild of the root. If it is the latter,

then its parent will be the second largest element.

Heap Property (Max heap property)

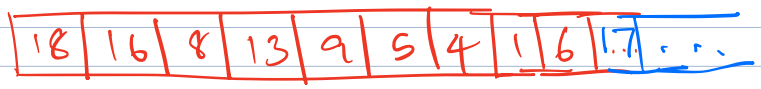
At each node i, $A[i] \geq A[j]$

when $j \in$ left child (i)
or right child (i)

Definition: A max heap is an array that satisfies both structural and max heap property at all nodes.

Note: In a max heap, for any node i, $A[i] \geq A[j]$ for all nodes j that are in the subtree rooted at i.

Also note: Heap property is NOT search property. There need not be a relation across left / right siblings.



Insert

→ Where to insert a new key?

→ Simplest to insert at the last position.

→ This causes violation of heap property.

→ This has to be fixed.

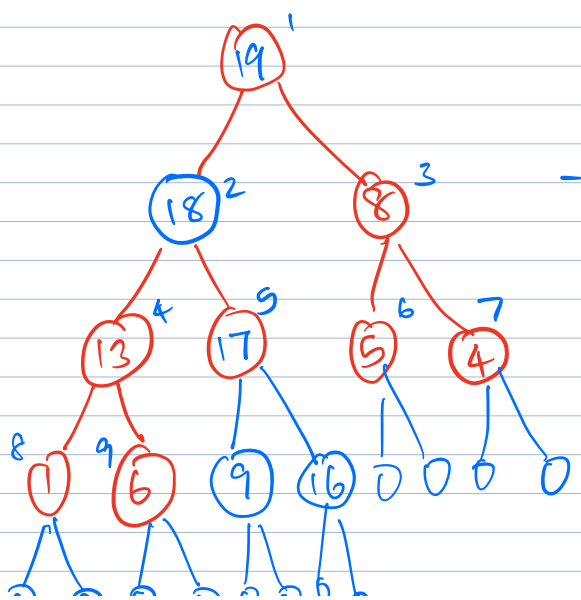
Insert (x)

→ Append x to the end of the array

→ Say x is inserted at A[j]

→ Can create a violation if the key at A[j] > key at parent

→ Fix by swapping. Repeat if necessary.



→ Insert 17.

No. of swaps $\leq$ Height : $\log n$

Time $= O(\log n)$

Correctness: Why is it still a max heap?

Replaced values are only higher in value. So other subtrees do not violate heap property.

Return max is $O(1)$ time.

Delete Max

→ Heap loses structure when deleting root. So we swap it with the last element.

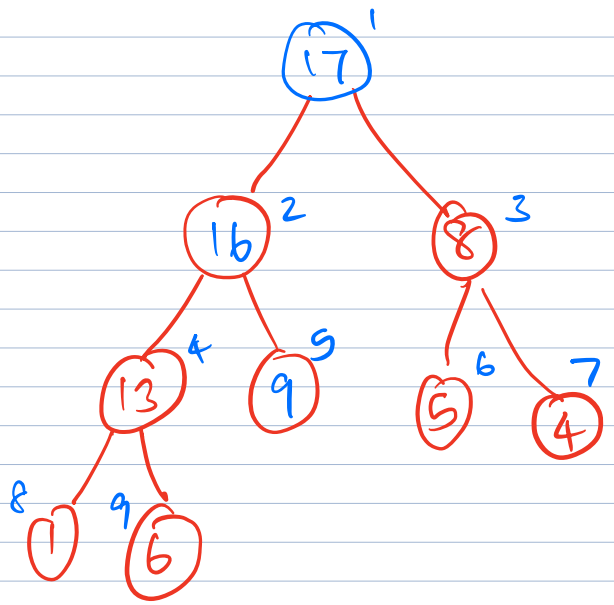
→ The new root may violate Max Heap property.

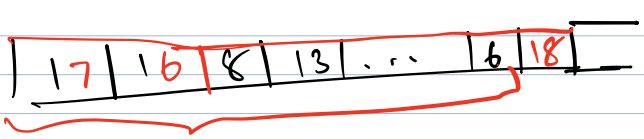
→ We repeatedly replace it with the larger child.

Max Heapify
Resolve downward conflicts



Time : $O(\log n)$





Heap Sort: Build Heap, with n elements

$O(n \log n)$ { And repeatedly delete max.
Can be done in-place.

↓

No need of extra storage.

→ How much time does it take to build heap?

Naive way = $n * \log n = n \log n \rightarrow n$

→ How much time is needed to do n delete max?

→ $n * \log n = n \log n$

Build Heap → can be done in $O(n)$

→ Repeated Insert takes $O(n \log n)$ time.

→ We can build it in $O(n)$ time

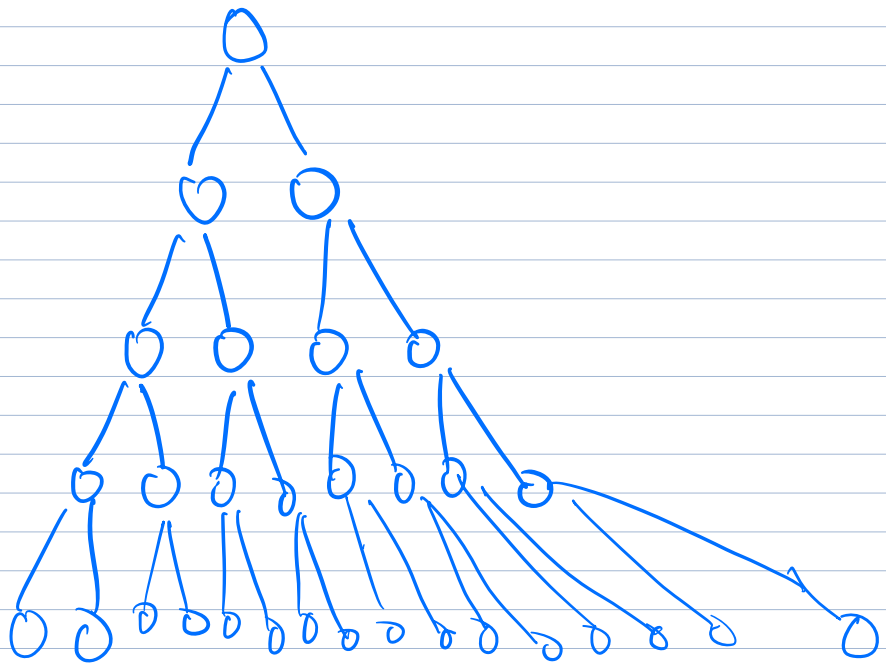
Given array $A[1 \ldots n]$.

For $i = n$ to 1

Heapify (i)

Naive analysis: This also takes $O(n \log n)$ time.

But many operations are small



No. of leafs = $\lceil n/2 \rceil$

No. of nodes at height one = $\lceil n/4 \rceil$

No. of nodes at height two = $\lceil n/8 \rceil$

Total number of warps needed $\Big\} \leq \left\lceil \frac{n}{4} \right\rceil * 1 + \left\lceil \frac{n}{8} \right\rceil * 2 + \left\lceil \frac{n}{16} \right\rceil * 3$

$$+ \cdots + \left\lceil \frac{n}{2^{(\log n + 1)}} \right\rceil * \log n$$

$$\leq \frac{n}{2} \cdot 1 + \frac{n}{4} \cdot 2 + \frac{n}{8} \cdot 3 + \cdots + \frac{n}{2^{\log n}} \cdot \log n$$

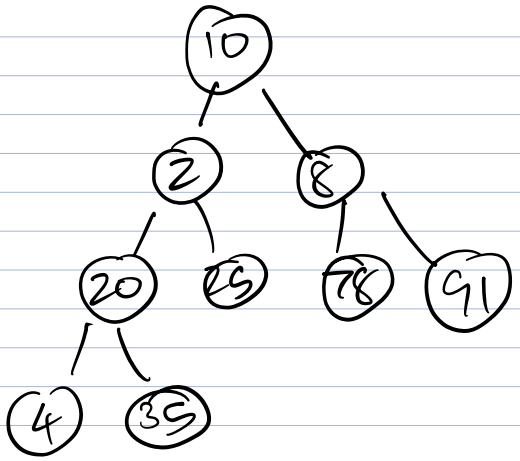
$$= n \underbrace{\left(\frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \cdots \frac{\log n}{2^{\log n}} \right)}_{S}$$

$$S - S/2 = 1/2 + 1/4 + 1/8 + \cdots \frac{1}{2^{\log n}}$$

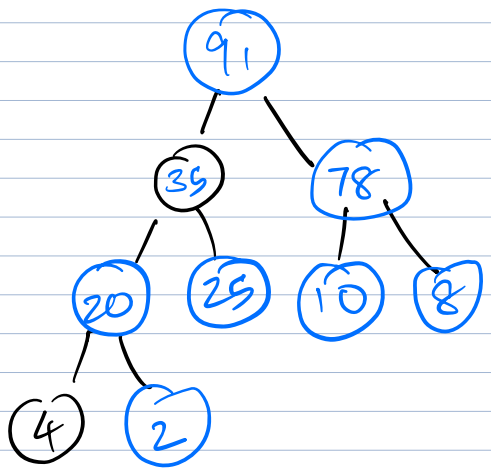
$$S/2 \leq 1$$

$$\leq 2n$$

$A = [10, 2, 8, 20, 25, 78, 91, 4, 35]$



$2^{h+1} - n$



$\left\lceil \frac{n}{2} \right\rceil$

If n is odd, say $2k+1$. Then the parent of n is k. So $k+1 = \lceil \frac{n}{2} \rceil$ nodes have no child.

Claim: This takes $O(n)$ time.

We are doing max heapify on $\lceil n/2 \rceil$ nodes. So we need

$\leq \lceil \frac{n}{2} \rceil * O(\log n)$ time

$= O(n \log n)$. Again this is not good.

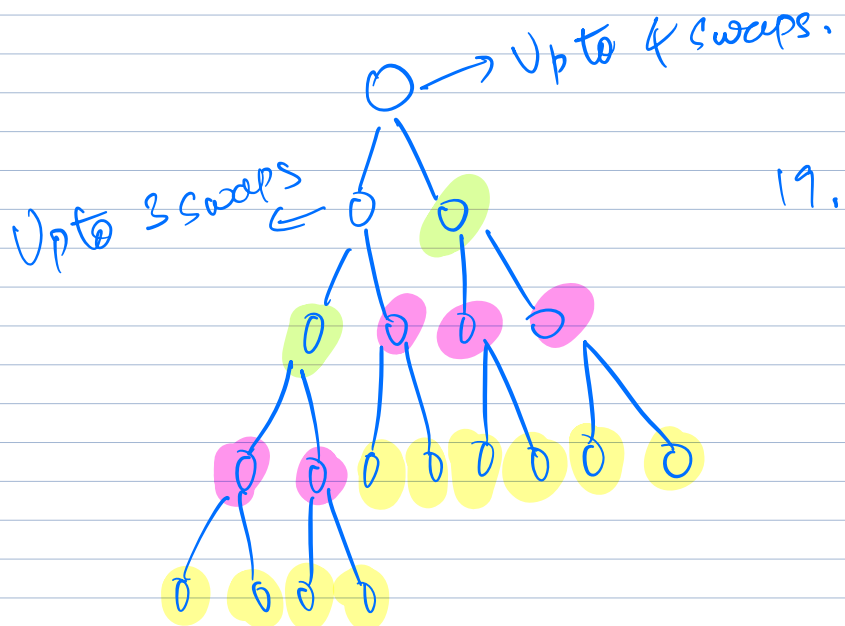
We can do a better analysis.

For $\lfloor \frac{n}{2} \rfloor$ nodes, no time is needed.

For $\lfloor \lfloor \frac{n}{2} \rfloor / 2 \rfloor$, we may need

to do ≤ 1 swap.

For $\frac{n}{k}$ nodes, we may need to do ≤ 2 swaps.



How many nodes have descendants upto l levels below? When $l=0$, these are nodes without children.

Nodes without children $= \left\lceil \frac{n}{2} \right\rceil$

Nodes with children, but have 1 level of descendant but not 2 levels $= \left\lceil \frac{n - \left\lceil \frac{n}{2} \right\rceil}{2} \right\rceil$

$= \left\lceil \frac{\left\lfloor \frac{n}{2} \right\rfloor}{2} \right\rceil \leq \left\lceil \frac{n}{4} \right\rceil$

Nodes with 2 levels of descendants but not 3 $= \left\lceil \frac{\left\lfloor \frac{\left\lceil \frac{n}{2} \right\rceil}{2} \right\rfloor}{2} \right\rceil \leq \left\lceil \frac{n}{8} \right\rceil$

Exercise: Verify the above calculation.